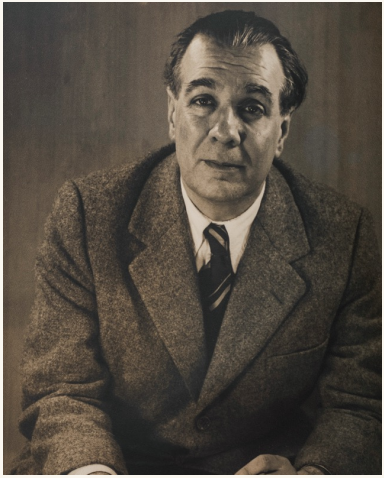
Two Engines

Memory, prediction, and the difference between them

Carlos Cotrini

ETH Zürich | AI in Industry

Week 2 — Spring 2026



"To think is to forget differences, generalize, abstract."

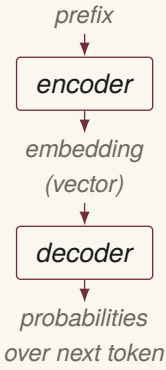
Jorge Luis Borges, *Funes the Memorious*, 1942

Last week, in language

You met the transformer. The summary we landed on:

An LLM ≈ encoder (builds meaning) + decoder (continues the sentence).

The *encoder* reads a prefix and produces an *embedding* — a vector summarising what the prefix says. The *decoder* takes that vector and writes a probability distribution over the next token.



Two halves of one machine

The encoder and decoder have older names.

- Encoder ≈ *compression*. A long sequence becomes a short vector.
- Decoder ≈ *prediction*. The vector becomes a bet about what comes next.

For ninety minutes today we will do each half explicitly — but for data that isn't language. *Customers, not words. Campaigns, not tokens.*

This framework is older than the transformer. It is from a 1948 paper by Shannon.

The promise

Machine learning is the engineering of two old theorems. *Compression* — the shortest faithful description of the past — and *prediction* — the best bet about the future — are two views of the same subject, governed by Shannon's 1948 theorems. We will build them in two halves and collapse them in the third.

"... a Map of the Empire whose size was that of the Empire, and which coincided point for point with it."

Jorge Luis Borges, *On Exactitude in Science*, 1946

Act I — Compression

The data: 21 readers, 16 books

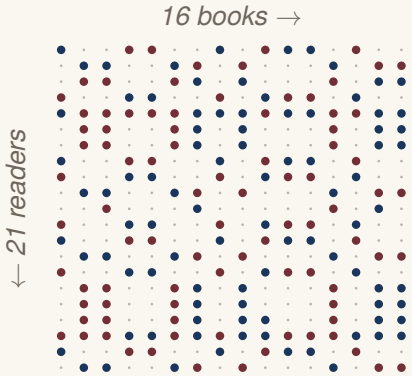
A small public library has kept a quiet record.

For each of *21 anonymous readers* and each of *16 books*, one cell records whether the reader liked the book, disliked it, or has not yet read it.

Three symbols. Three hundred and thirty-six cells.

No names, no genres, no author photographs — only the matrix.

Raw data



Three hundred and thirty-six cells. Can your brain remember them?

Now switching to:

01_block-diagonal-puzzle.html

The geometry was always there

Both axes are old library science: Dewey decimal (1876) gives Fiction vs. Nonfiction; Flesch (1948) and Lexile (1989) give Light vs. Demanding.

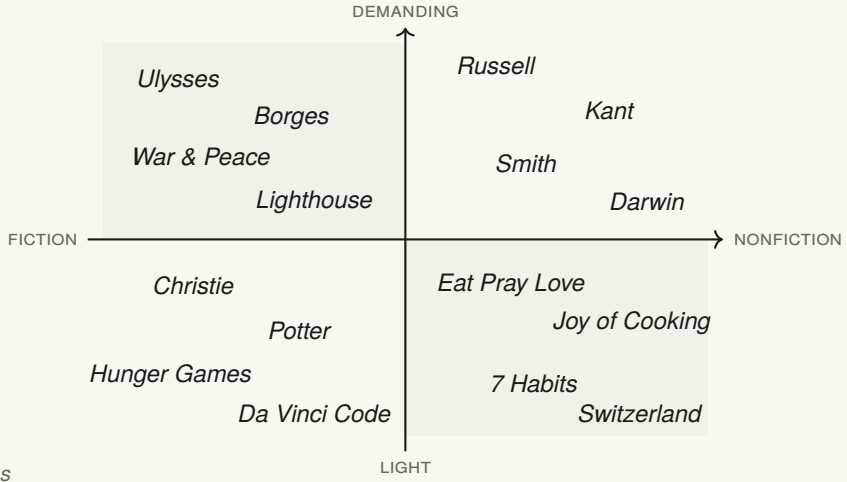
When the matrix is sorted by hand, two axes appear:

- *Fiction* versus *Nonfiction*
- *Light* versus *Demanding*

Neither axis was invented for this dataset. They are how librarians and educators have sorted books for a century.

We did not impose them. We *recovered* them.

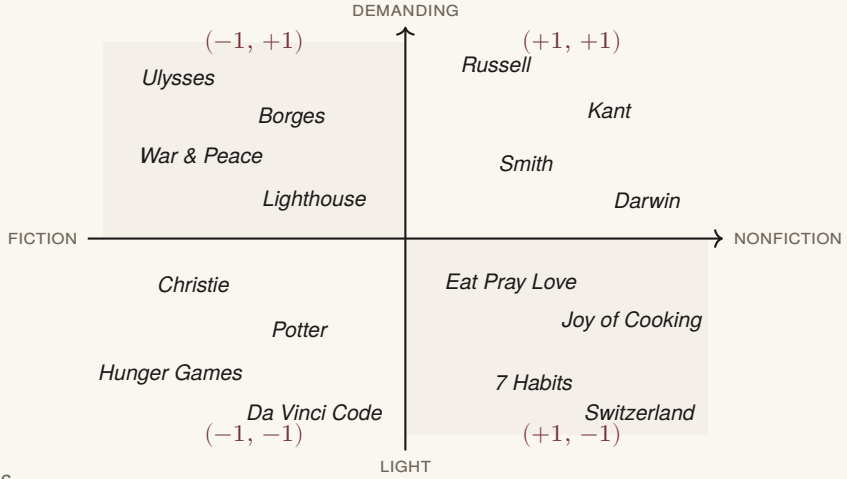
Sixteen books, four quadrants



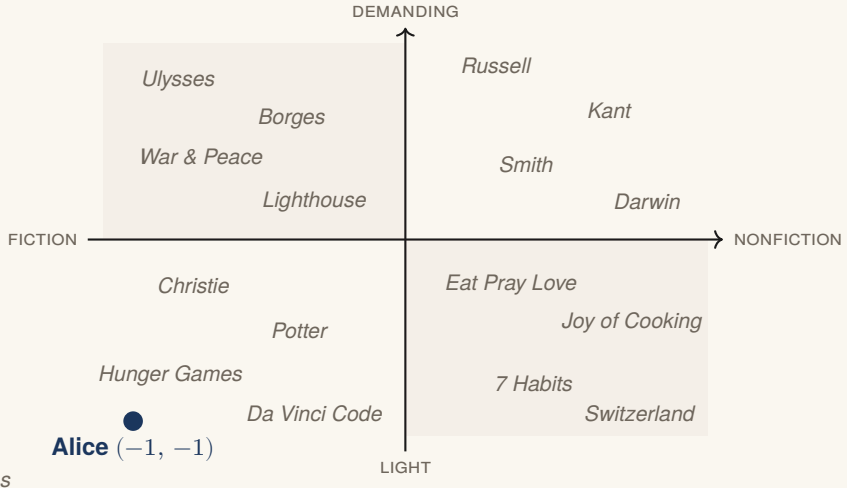
Now switching to:

02_latent-axes-lab.html

Each book gets two numbers



And readers, too



How much will Alice like Russell?

Alice	**Russell**
light fiction	demanding nonfiction
$(-1,\ -1)$	$(+1,\ +1)$

$$\hat{r}_{\text{Alice, Russell}} \;=\; (-1)\cdot(+1) \;+\; (-1)\cdot(+1) \;=\; -2$$



Alice will not finish Russell.

How much will Alice like Potter?

Alice	**Potter**
light fiction	light fiction
$(-1,\ -1)$	$(-1,\ -1)$

$$\hat{r}_{\text{Alice, Potter}} = (-1)\cdot(-1) + (-1)\cdot(-1) = +2$$

✓

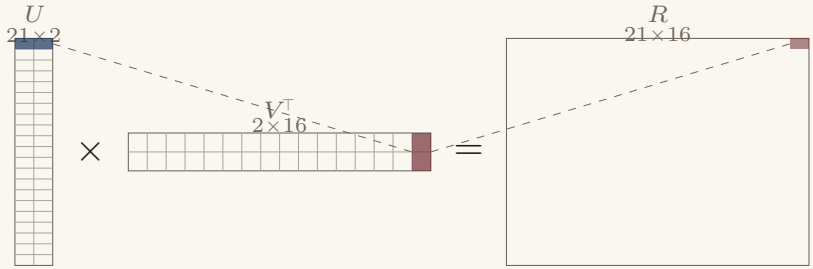
Alice will love Potter. Same formula, opposite sign.

Preference is a dot product

$$\hat{r}_{ui} = u_i^\top v_j$$

The reader's vector and the book's vector live in the same space; their inner product is the predicted preference. Two coordinates was the easy case — the formula keeps its shape in any dimension.

Stack and multiply



21 readers in U, 16 books in V, their product *is* R. Each cell is one dot product — the highlighted one is Alice $\times$ Russell.

The matrix product, in numbers

U

Alice	-1	-1
Eleni	-1	$+1$
Maya	$+1$	$+1$

$\times$

$V^\top$

Potter	Borges	Russell
-1	-1	$+1$
-1	$+1$	$+1$

$=$

R

Potter	Borges	Russell
$+2$	0	-2
0	$+2$	0
-2	0	$+2$

Each entry of R is one dot product. *Positive* on the diagonal: each reader sits with her favourite book. *Negative* on the anti-diagonal: each reader far from the wrong book. *Zero* where the two are perpendicular.

But we only had R

We never observed U or V directly. The library's ledger has only R — the rating matrix, three hundred and thirty-six cells of symbols.

Yet the matrix is the product of two skinny ones. So the question is:

Given R alone, can we recover U and V?

This is the *matrix factorization* problem. The next demo gives one way to solve it.

Now switching to:

spectral-microscope.html

Eckart & Young (1936)[1]

> *Among all rank-k approximations of any matrix, the one given by the top-k singular vectors is optimal in squared error.*

No algorithm beats it. No clever trick beats it. No future architecture beats it.

The curve you just saw is not an empirical observation. It is the graph of a theorem ninety years old.

Now switching to:

customer-urns-carousel.html

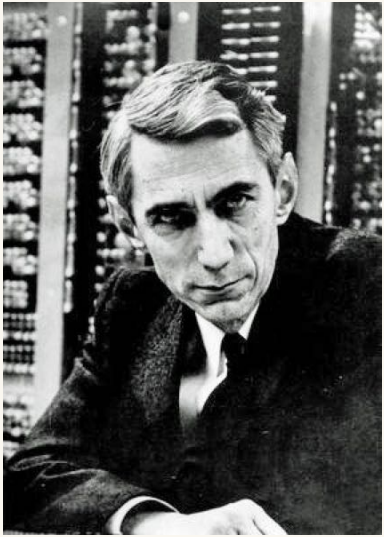
Now switching to:

07_embeddings-that-learn.html

End of Act I

We compressed a matrix. We named the act.

The names — *entropy*, *rank*, *factorization* — were old words from 1936 and 1948.



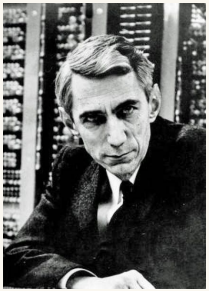
"The fundamental problem of communication is that of reproducing at one point either exactly or approximately a message selected at another point."

Claude E. Shannon, 1948

Interlude

Shannon, 1948

In 1948, Claude Shannon published *A Mathematical Theory of Communication* — a paper with two theorems at its core. The *source coding theorem* fixes a floor on how compactly any sequence of symbols can be described; the *channel coding theorem* fixes a ceiling on how reliably they can be transmitted through noise. For the next seven minutes we restate the first of these in the only vocabulary this lecture admits: urns.



How unpredictable is the next ball?

An urn with bias p produces a stream of white and yellow draws. When $p = \frac{1}{2}$ every draw is a coin flip — maximal surprise. When $p = 0.99$ almost every draw is white — the sequence is nearly redundant. *Entropy* is the name we give to this gradient between certainty and surprise. Let us feel it before we write it down.

Now switching to:

entropy-mountain.html

The entropy of an urn

$$H(p) = -p \log_2 p - (1-p) \log_2 (1-p)$$

No compression scheme can describe an urn's draws in fewer than $H(p)$ bits per ball. Eckart–Young was the floor for compressing a matrix; entropy is the floor for compressing a sequence.

Recsys was compression. Bandits will be prediction.
Same urns, different question.

Act II — Prediction

The bandit problem

A show of hands: who has run an A/B test?

Three marketing campaigns. Three urns. Each urn hides a conversion rate μ_i that you do not know.

- Each round, you pick one urn and draw a single ball.
- A white ball is a conversion; a grey ball is nothing.
- Your goal: *maximise total conversions* over T rounds.

Every draw is information *and* payoff at the same time. That double role is what makes the problem interesting.

The empirical estimator

$$\hat{p}_n = \frac{x_1 + x_2 + ... + x_n}{n}$$

Average and pick the best — the audience's natural instinct.

Explore vs exploit

Two pressures pull in opposite directions.

- *Exploit*: commit to the urn that looks best so far, and harvest its conversions.
- *Explore*: keep sampling the others, in case the leader is only ahead by luck.

Exploit too early and you may have crowned the wrong urn. Explore too long and you waste rounds on losers. That tension has a name, and the next twenty minutes are about it.

Naïve: estimate then exploit

A first attempt:

1. Explore each urn n times, round-robin.
2. Commit to the urn with the highest empirical mean $\hat{p}_n$.
3. Exploit that urn forever after.

Two drawbacks:

- If n is too small, the empirical means are noisy and you commit to the wrong urn.
- If n is too large, you waste rounds on losers before exploiting.

The choice of n is itself the problem.

Now switching to:

three-urn-casino.html

ε-greedy

$$A_t \;=\; \begin{cases} \text{random urn}, & \text{w.p. } \varepsilon \\ \arg\max_i \hat{p}_{n_i}, & \text{w.p. } 1-\varepsilon \end{cases}$$

Most of the time, exploit; a small fraction, explore. ε is the dial between the two pressures. — Sutton & Barto (1998/2018).

Race the algorithms

Five strategies, the same three urns, the same hidden biases:

- Pure random — no learning at all.
- Pure greedy — exploit from round one ($\varepsilon = 0$).
- ε-greedy at three values of ε.

Watch the cumulative-regret curves. The slope is the whole story: random and greedy grow regret *linearly*; a well-tuned ε-greedy grows it *sublinearly*.

Now switching to:

algorithm_stadium.html

How many users do you need?

This is what your statistician means when she says "we need a million users to A/B-test this."

It depends on the gap.

Two urns with conversion rates 0.50 and 0.40 are easy to tell apart. Two urns at 0.50 and 0.49 are nearly indistinguishable; separating them takes *far* more samples.

The required sample size scales roughly as $1/(p_1 - p_2)^2$. Halve the gap, and you need four times the data.

Now switching to:

distinguishability-dial.html

Lai & Robbins (1985)

$$\liminf_{T\to\infty} \frac{R_T}{\log T} \;\geq\; \sum_{i\neq i^*} \frac{\mu^* - \mu_i}{\mathrm{KL}(\mu_i \,\|\, \mu^*)}$$

No algorithm beats this rate. The hardest thing is telling apart distributions that look alike — and the cost is in KL divergence, the same Shannon used in 1948.

Two halves, one ceiling

Eckart–Young bounded compression.

Lai–Robbins bounds prediction.

The two halves of this lecture share one ceiling — and we have not yet named the quantity in which both ceilings are written. That is the Epilogue.



“Compression is comprehension.”

Marcus Hutter, on the spirit of his prize

Epilogue — Duality

A perfect predictor is a perfect compressor

Two halves of one subject.

- A *forecaster* assigns $\hat{p}$ to the next symbol; her loss is $-\log \hat{p}$ per draw.
- An *encoder* writes short codewords for likely symbols; his cost is bits per draw.
- Shannon (1948) proved these are the same number, bottoming out at $H(p)$.

Hutter calls this *compression is comprehension*; Delétang et al. (2023) confirm it: a text-trained LM compresses ImageNet better than PNG.

Now switching to:

compression-prediction-twin.html

Floor + inefficiency

$$\underbrace{-\,\mathbb{E}_p[\log \hat{p}]}_{\text{cross-entropy}} = \underbrace{H(p)}_{\text{Shannon's floor}} + \underbrace{\mathrm{KL}(p \,\|\, \hat{p})}_{\text{your inefficiency}}$$

Recsys: KL is the gap between your factorization and the truth. Bandits: KL is the cost of mistaking one urn for another. Compression: KL is the bits you waste because your predictor is imperfect. One quantity, three roles.

Funes remembered everything and could think nothing.

Machine learning forgets almost every-
thing — and that is why it understands.

References, I

1. J. L. Borges, “Funes el memorioso,” *Ficciones*, 1944.
2. C. Eckart and G. Young, “The approximation of one matrix by another of lower rank,” *Psychometrika* 1, 211–218, 1936.
3. C. E. Shannon, “A mathematical theory of communication,” *Bell Syst. Tech. J.* 27, 379–423, 1948.
4. T. L. Lai and H. Robbins, “Asymptotically efficient adaptive allocation rules,” *Adv. Appl. Math.* 6, 4–22, 1985.

References, II

5. R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.
6. M. Hutter, *Universal Artificial Intelligence*, Springer, 2005.
7. G. Delétang *et al.*, "Language modeling is compression," arXiv:2309.10668, 2023.

Image credits. Borges (1951): Grete Stern, public domain. Shannon: Konrad Jacobs, Mathematisches Forschungsinstitut Oberwolfach, CC-BY-SA 2.0 DE. Hutter: Wikimedia Commons (Marcus Hutter), CC-BY-SA. All retrieved from Wikimedia.